

Clasificación de Residuos

Computer Vision 2025

Maestría en Ciencia de Datos y Aprendizaje Automático

Joana Auriello

Pablo Molina

[Introduccion](#)

[Resumen](#)

[Descripción del problema / clasificación residuos](#)

[Objetivo general / objetivos específicos](#)

[Bases utilizadas](#)

[Transformaciones/augmentation pipeline](#)

[Modelos](#)

[Resnet50 - baseline](#)

[Adaptación del modelo ResNet50 a la clasificación de residuos](#)

[Resultados Resnet50 con augmentation](#)

[Resultados Resnet50 sin augmentation](#)

[Comparación entre ResNet50 con y sin augmentation](#)

[EfficientNet - mejora sobre resnet50](#)

[YOLO](#)

[Modelo base y librería utilizada](#)

[Arquitectura YOLOv8 para clasificación](#)

[Ajuste del modelo a nuestro caso específico](#)

[Configuración del entrenamiento](#)

[Entrenamiento y ajuste final](#)

[Resultados YOLO sin Augmentation](#)

[Resultados comparativos - Tablita si se puede: por categoria, modelo, val_accuracy](#)

[Conclusiones y aspectos de mejora](#)

[Anexo/Recursos \(links a articulos , diagramas, google collab, repositorios importantes\)](#)

Introduccion

La clasificación automática de residuos es un problema creciente tanto en complejidad como en importancia. La correcta separación de materiales reciclables y no reciclables contribuye a la reducción de desechos, al aumento de las tasas de reciclaje y a la optimización de procesos industriales. En este contexto, los modelos de visión por computadora permiten automatizar la

identificación de categorías de residuos a partir de imágenes, mejorando la eficiencia y reduciendo el error humano.

Este proyecto explora diversas arquitecturas modernas de deep learning: ResNet50, EfficientNet y YOLOv8, aplicadas al dataset TrashNet y sus variantes extendidas. Nuestro objetivo es evaluar su desempeño, identificar sus limitaciones y proponer adaptaciones que mejoren la capacidad de clasificación.

Resumen

En este proyecto se desarrolla y evalúa un sistema de clasificación automática de residuos urbanos utilizando técnicas de visión por computadora y aprendizaje profundo. Con el fin de construir un conjunto de datos amplio y representativo, se recopilaron imágenes provenientes de diversos datasets públicos disponibles en línea, las cuales fueron seleccionadas, integradas, depuradas y reclasificadas manualmente por el equipo. El resultado es un dataset unificado de aproximadamente 59.000 imágenes, distribuidas en categorías de residuos relevantes para el reciclaje: bio organico, envase plasticos, papel y carton, Reciclables Varios cigarro, .Reciclables Varios Electroincos, Reciclables Varios Madera, Reciclables Varios Metal, Reciclables Varios Otros, Reciclables Varios Plastico, Reciclables Varios Textiles, Reciclables Varios Vidrio, Todo lo demas.

El conjunto de datos se divide de forma estratificada en tres particiones independientes: entrenamiento (80 %), validación (10 %) y prueba (10 %). Sobre esta base se entrenan y comparan cuatro modelos de clasificación basados en transferencia de aprendizaje: ResNet50 y EfficientNet, cada uno en dos variantes, con y sin data augmentation. El modelo sin augmentation actúa como baseline, mientras que las variantes con augmentation aplican transformaciones geométricas y fotométricas online para mejorar la capacidad de generalización del modelo frente a variaciones de iluminación, perspectiva y escala.

El entrenamiento se realiza en dos fases: (i) una primera etapa donde se entrena únicamente la cabeza clasificadora con el backbone congelado, y (ii) una segunda etapa de fine-tuning donde se descongelan las capas superiores del modelo y se ajustan con una tasa de aprendizaje reducida. Se utilizan técnicas de regularización como early stopping, reduce-on-plateau y dropout, así como épocas acotadas mediante un número controlado de steps per epoch, optimizando el entrenamiento en entornos de cómputo limitados (Google Colab, GPU gratuita).

La evaluación final se realiza exclusivamente sobre el conjunto de prueba independiente, utilizando métricas estándar como accuracy, precision, recall y matrices de confusión por clase. Finalmente, se discuten las diferencias de desempeño entre las arquitecturas evaluadas, el impacto de la augmentation en la capacidad de generalización y el potencial de estos modelos para su integración futura en sistemas automatizados de clasificación de residuos en Uruguay.

Descripción del problema / clasificación residuos

La correcta clasificación de residuos sólidos urbanos es un paso crítico para la gestión ambiental y el reciclaje. En Uruguay, existen normativas y campañas que promueven la separación en origen (orgánicos, reciclables, sanitarios, etc.), pero en la práctica persisten dificultades: participación ciudadana limitada, confusión respecto a qué residuos son reciclables, infraestructura heterogénea y procesos de clasificación manual que pueden ser costosos y poco eficientes.

Desde el punto de vista de la visión por computadora, el problema se plantea como una tarea de clasificación multiclase de imágenes: dado un recorte de imagen que contiene un objeto (por ejemplo, una botella de plástico o una lata de metal), el modelo debe asignarla a una de las categorías predefinidas. En este proyecto se trabaja con un esquema de clases simplificado y alineado con criterios de reciclaje habituales:

1. bio organico
2. envase plasticos
3. papel y carton
4. Reciclables Varios cigarro
5. .Reciclables Varios Electrónicos
6. Reciclables Varios Madera
7. Reciclables Varios Metal
8. Reciclables Varios Otros
9. Reciclables Varios Plastico
10. Reciclables Varios Textiles
11. Reciclables Varios Vidrio
12. Todo lo demas (no reciclable)

Estas categorías capturan los flujos de residuos más relevantes en contextos urbanos y son coherentes con los sistemas de clasificación implementados en Uruguay y en otros países de la región.

Objetivo general / objetivos específicos

Nuestro objetivo general es desarrollar y evaluar un modelo de visión por computadora basado en aprendizaje profundo capaz de clasificar imágenes de residuos en categorías relevantes para el reciclaje, analizando su potencial aplicación en el contexto de gestión de residuos en Uruguay.

Objetivos específicos

1. **Relevamiento y selección de datos**
Analizar distintos datasets públicos de residuos disponibles en línea, comparando su

calidad, tamaño, estructura de clases y licencias, para seleccionar un conjunto de datos adecuado para el entrenamiento del modelo.

2. **Diseño del pipeline de preprocesamiento y augmentación**

Implementar un pipeline de preprocesamiento de imágenes (redimensionado, normalización) y, cuando sea pertinente, data augmentation para aumentar la diversidad efectiva de los datos de entrenamiento.

3. **Entrenamiento y validación de un modelo base (baseline)**

Entrenar una arquitectura ResNet50 y EfficientNet sin augmentation como modelo base, utilizando transferencia de aprendizaje, y evaluar su desempeño mediante accuracy, precision, recall y matrices de confusión. Identificar las principales fuentes de error de clasificación.

4. **Entrenamiento de un modelo optimizado con augmentación y fine-tuning**

Entrenar una variante de ResNet50 y EfficientNet con augmentation online y una fase de fine-tuning del backbone, comparando sistemáticamente sus resultados con el baseline.

5. **Evaluación en un conjunto de prueba independiente**

Separar los datos en conjuntos de entrenamiento, validación y prueba, de forma estratificada, y utilizar el conjunto de prueba exclusivamente para la evaluación final del modelo.

6. **Discusión del impacto potencial**

Analizar, a nivel teórico, cómo un sistema de este tipo podría integrarse en políticas y dispositivos de clasificación de residuos en Uruguay, discutiendo limitaciones técnicas y desafíos prácticos.

Bases utilizadas

Se utilizaron versiones del dataset **TrashNet**, que incluye imágenes clasificadas en categorías como *plastic*, *metal*, *glass*, *paper*, *cardboard* y *trash*. El conjunto fue dividido en sets de entrenamiento, validación y prueba manteniendo proporciones equilibradas y asegurando que las imágenes usadas para evaluación no fueran vistas durante el entrenamiento.

Materiales que podés depositar:



Artículos electrónicos pequeños de uso doméstico:
celulares, computadoras, cartuchos de tinta, pilas, baterías de litio, lámparas y tubos fluorescentes, etc.



Materiales reciclables (limpios, secos y compactados):
papel, cartón y plásticos.



Envases de vidrio



Metales



Aceite de uso doméstico usado



Ropa, calzado y textiles



Medicamentos vencidos y sus envases



Residuos compostables:
Restos de frutas y verduras crudas, yerba, café e infusiones.



Se reciben únicamente materiales de origen domiciliario.



No se reciben Materiales de origen comercial o industrial

Te recordamos que para los residuos de gran tamaño como muebles, electrodomésticos, podas y escombros domiciliarios, podés solicitar su retiro de forma gratuita a través del ☎ 092 250 260

Ahora cada Municipio tienen su día, conocé el tuyo a través del siguiente link montevideo.gub.uy/servicio-de-recoleccion-de-residuos-grandes



Transformaciones/augmentation pipeline

El pipeline de datos se implementó íntegramente en TensorFlow/Keras. A partir del archivo CSV consolidado con aproximadamente 59.000 imágenes y sus respectivas etiquetas, se realizó una división estratificada por clase en tres subconjuntos independientes: 80 % para entrenamiento, 10 % para validación y 10 % para prueba. La estratificación garantiza que la distribución de clases se mantenga equilibrada en los tres conjuntos, evitando sesgos que puedan afectar el desempeño del modelo. El conjunto de entrenamiento se utilizó para ajustar los pesos de los modelos; el conjunto de validación permitió seleccionar hiperparámetros, aplicar early stopping y elegir los mejores checkpoints sin actualizar pesos; y el conjunto de

prueba se mantuvo completamente aislado, usandose una única vez al finalizar todo el proceso para evaluar objetivamente el modelo seleccionado.

En todos los modelos se aplicó un preprocesamiento común, consistente en redimensionar cada imagen a 224×224 píxeles en formato RGB, tamaño estándar para arquitecturas preentrenadas como ResNet50 y EfficientNet. Las imágenes se convirtieron a tensores float32 y se normalizaron a un rango [0, 1] mediante `rescale=1./255`. Adicionalmente, se habilitó `PIL.ImageFile.LOAD_TRUNCATED_IMAGES = True` con el objetivo de manejar adecuadamente imágenes truncadas o dañadas presentes en el dataset, evitando fallos durante la carga.

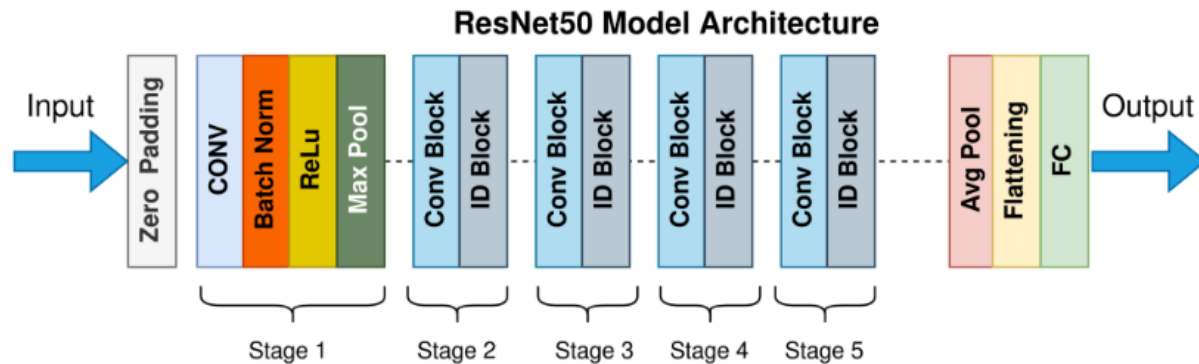
Para los modelos baseline sin augmentation(ResNet50 y EfficientNet), tanto el generador de entrenamiento como los de validación y prueba utilizaron el mismo transformador `ImageDataGenerator(rescale=1./255)`. En esta configuración no se aplican transformaciones adicionales, lo que permite evaluar el rendimiento de cada arquitectura únicamente a partir de los datos originales, sin alteraciones artificiales.

Por otro lado, en los modelos con augmentation, se incorporó un esquema de transformaciones online aplicado exclusivamente al conjunto de entrenamiento. Se emplearon operaciones como volteo horizontal, rotaciones de hasta 15°, zoom de $\pm 20\%$ y variaciones moderadas de brillo. Estas transformaciones simulan condiciones reales de captura, como cambios de iluminación, orientación o distancia, y contribuyen a mejorar la capacidad de generalización del modelo. Los generadores de validación y prueba, en contraste, sólo aplican `rescale=1./255`, evitando cualquier tipo de augmentation para asegurar evaluaciones consistentes.

Modelos

Para este proyecto seleccionamos tres arquitecturas de referencia utilizadas en visión por computadora: ResNet50, EfficientNet y YOLO. Estas redes han demostrado un excelente desempeño en tareas de clasificación de imágenes, combinando la potencia de la transferencia de aprendizaje con la eficiencia computacional necesaria para aplicaciones reales. La elección de estas arquitecturas se basa en criterios de robustez, disponibilidad de implementaciones estables en TensorFlow/Keras/Ultralytics, y evidencia en la literatura que muestra su efectividad en problemas similares de clasificación de residuos. A efectos comparativos, las arquitecturas se entrena en dos variantes: una versión baseline sin augmentation y una versión con augmentation, permitiendo analizar el impacto de esta técnica sobre la capacidad de generalización del modelo.

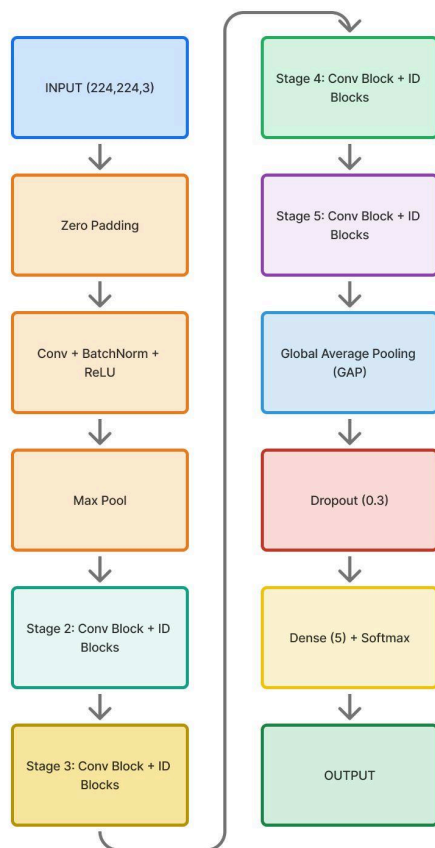
Resnet50 - baseline



[Resnet-50 Model architecture](#)

ResNet50 es una red neuronal convolucional profunda de 50 capas. Su principal innovación radica en el uso de bloques residuales con skip connections, los cuales permiten que la red aprenda funciones residuales en lugar de mapeos directos. Estas conexiones facilitan el flujo del gradiente durante el entrenamiento, resolviendo el problema de la degradación en redes muy profundas y permitiendo entrenar modelos con cientos de capas. ResNet50 fue originalmente entrenada en el benchmark ImageNet, lo que la convierte en una candidata ideal para tareas basadas en transferencia de aprendizaje debido a la riqueza de representaciones visuales que aprende en ese contexto.

Adaptación del modelo ResNet50 a la clasificación de residuos



ResNet-50 — Explanation of Each Step

1. Input (224 × 224 × 3)

- The network receives a standard RGB image.
- All images are resized to 224×224 to keep the architecture consistent.

2. Stem: 7×7 Convolution + MaxPool

7×7 Conv, 64 filters, stride 2

- Quickly reduces spatial size (224 → 112) while capturing broad visual patterns like edges and color gradients.

- Large receptive field early helps the network understand context.

3×3 MaxPool, stride 2

- Further reduces resolution (112 → 56).
- Keeps the strongest activations—highly discriminative for early visual features.

3. Stage 1 (Conv2_x): 3 Bottleneck Blocks, Output 256

Each bottleneck block = 1×1 → 3×3 → 1×1 convolutions.

Purpose of each step:

- 1×1 reduce: shrink channel depth to make the block efficient.
- 3×3 conv: main spatial feature extraction.
- 1×1 expand: restore the channel depth for richer representations.

Residual connection:

- Input is added to output → prevents vanishing gradients, stabilizes training.

Output feature map: 56 × 56 × 256

4. Stage 2 (Conv3_x): 4 Bottleneck Blocks, Output 512

- First block uses stride 2 to downsample (56 → 28).
- Channels are increased to 512.
- This stage learns texture patterns, shapes, and mid-level semantic structures.

Output: 28 × 28 × 512

5. Stage 3 (Conv4_x): 6 Bottleneck Blocks, Output 1024

- Largest and most computationally heavy stage.
- First block downsamples (28 → 14).
- Builds rich semantic features: object parts, contours, structure.

Output: 14 × 14 × 1024

6. Stage 4 (Conv5_x): 3 Bottleneck Blocks, Output 2048

- Deepest features.
- First block downsamples (14 → 7).
- Learns high-level semantic concepts: object identity, category cues.

Output: 7 × 7 × 2048

7. Global Average Pooling (GAP)

- Converts each of the 2048 feature maps into one number.
- Collapses 7×7 → 1×1, preserving global semantics.
- Reduces parameters dramatically → less overfitting.

Output: 2048-dim vector

8. Fully Connected Layer + Softmax

- Final classification layer.
- Produces logits → Softmax transforms them into class probabilities.

Luego de establecer el baseline, se procedió a adaptar ResNet50 para mejorar su capacidad de generalización en el dominio específico de residuos. Para ello, se reemplazó completamente la cabeza de clasificación original y se definió un clasificador personalizado más adecuado al dataset utilizado.

La nueva cabeza del modelo incluye un bloque de Global Average Pooling (GAP), seguido de una capa totalmente conectada, un Dropout del 30% para regularización, y finalmente una capa softmax con salida correspondiente al número de clases del conjunto de datos. Esta arquitectura permite aprovechar las características extraídas por ResNet50, al tiempo que reduce el riesgo de sobreajuste y mejora la discriminación entre categorías visualmente similares.

Para evaluar adecuadamente el impacto de la regularización y la robustez del modelo, se realizaron dos entrenamientos separados:

1. Entrenamiento sin augmentations

Este experimento utiliza únicamente las imágenes originales del dataset. Permite medir

el rendimiento “puro” del modelo sin variaciones sintéticas, y sirve como referencia para entender en qué medida el modelo puede memorizar patrones específicos del conjunto de entrenamiento.

2. Entrenamiento con augmentations

En una segunda instancia, se entrenó el modelo incorporando un pipeline de aumentación moderado, que incluye rotaciones, flips horizontales, ajustes de brillo/contraste y normalización estándar. El objetivo fue simular condiciones reales de captura, aumentar la diversidad del dataset e incentivar al modelo a aprender representaciones más invariantes y generalizables.

La comparación entre ambos entrenamientos permite analizar cómo responde el modelo al aumento de diversidad y qué tan sensible es a las condiciones originales del dataset. Esta evaluación es clave para seleccionar la configuración más adecuada para escenarios reales de clasificación.

El entrenamiento se realiza en dos fases:

1. Fase 1: Entrenamiento de la cabeza clasificadora

El backbone permanece congelado (`trainable = False`) y solo se entrenan las capas superiores recién añadidas. Se utiliza el optimizador Adam con una tasa de aprendizaje de `1e-3`, acompañada de early stopping, reduce-on-plateau y guardado automático del mejor checkpoint. Para optimizar el tiempo de entrenamiento en Google Colab, cada época se limita mediante un número reducido de steps per epoch, manteniendo una muestra representativa del dataset sin necesidad de recorrer todos los lotes.

Model: "functional"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 224, 224, 3)	0
resnet50 (Functional)	(None, 7, 7, 2048)	23,587,712
global_average_pooling2d (GlobalAveragePooling2D)	(None, 2048)	0
dropout (Dropout)	(None, 2048)	0
dense (Dense)	(None, 12)	24,588

Total params: 23,612,300 (90.07 MB)

Trainable params: 24,588 (96.05 KB)

Non-trainable params: 23,587,712 (89.98 MB)

2. Fase 2: Fine-tuning del backbone

Una vez entrenada la cabeza, se descongela aproximadamente el 30 % superior de las capas del backbone para permitir un ajuste fino sobre el dominio específico de residuos. La tasa de aprendizaje se reduce a $1e-5$ para evitar modificaciones bruscas en los pesos preentrenados. Esta fase mejora la capacidad del modelo para capturar matices visuales particulares del dataset, como texturas de residuos orgánicos o reflejos en materiales reciclables.

```
Loading best Phase 2 model for experiment: resnet50_with_aug_v2
from: /content/drive/MyDrive/Colab Notebooks/ComputerVision/experiments/resnet50_with_aug_v2/checkpoints/resnet50_with_aug_v2_phase2_best.keras
Backbone name: resnet50
Total backbone layers: 175
Trainable backbone layers in Phase 2: 53
Model: "functional"
```

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 224, 224, 3)	0
resnet50 (Functional)	(None, 7, 7, 2048)	23,587,712
global_average_pooling2d (GlobalAveragePooling2D)	(None, 2048)	0
dropout (Dropout)	(None, 2048)	0
dense (Dense)	(None, 12)	24,588

Total params: 23,612,300 (90.07 MB)

Trainable params: 17,238,028 (65.76 MB)

Non-trainable params: 6,374,272 (24.32 MB)

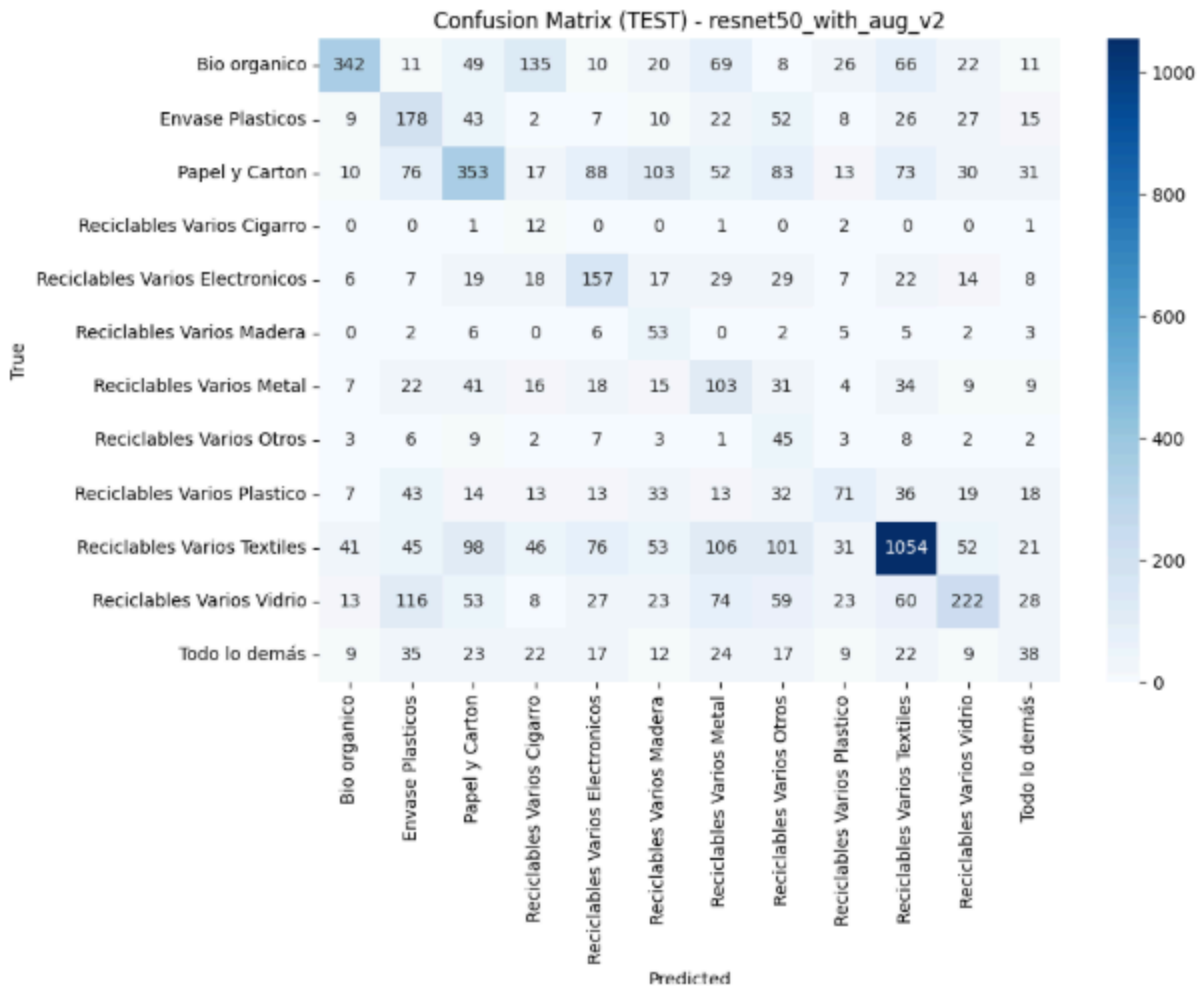
El resultado es un modelo que combina la generalización adquirida por ResNet50 en ImageNet con una especialización orientada a la clasificación de residuos urbanos.

Resultados Resnet50 con augmentation

Classification Report (TEST):

	precision	recall	f1-score	support
Bio organico	0.77	0.44	0.56	769
Envase Plasticos	0.33	0.45	0.38	399
Papel y Carton	0.50	0.38	0.43	929
Reciclables Varios Cigarro	0.04	0.71	0.08	17
Reciclables Varios Electronicos	0.37	0.47	0.41	333
Reciclables Varios Madera	0.15	0.63	0.25	84
Reciclables Varios Metal	0.21	0.33	0.26	309
Reciclables Varios Otros	0.10	0.49	0.16	91
Reciclables Varios Plastico	0.35	0.23	0.28	312
Reciclables Varios Textiles	0.75	0.61	0.67	1724
Reciclables Varios Vidrio	0.54	0.31	0.40	706
Todo lo demás	0.21	0.16	0.18	237
accuracy			0.44	5910
macro avg	0.36	0.44	0.34	5910
weighted avg	0.55	0.44	0.48	5910

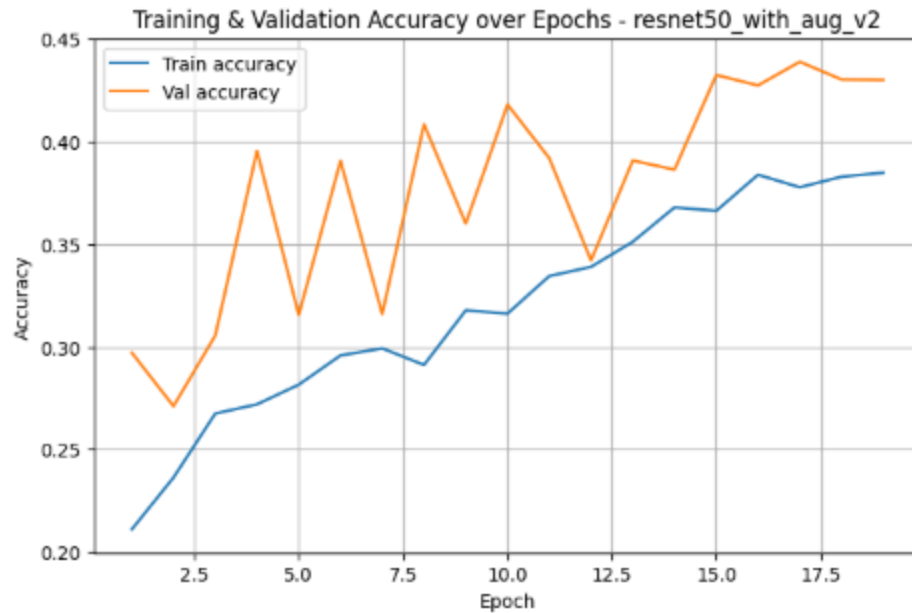
TEST classification report saved to: /content/drive/MyDrive/Colab Notebooks/ComputerVision/experiments/resnet50_with_aug_v2/resnet50_with_aug_v2_TEST_classification_report.txt



El accuracy global alcanzado es 0.44, acompañado por un macro F1-score de 0.44, lo cual evidencia un rendimiento moderado pero con una alta variabilidad entre clases.

A partir del classification report se observa que algunas clases presentan métricas significativamente más altas que otras. Por ejemplo, Bio orgánico obtiene un F1-score de 0.56, Papel y cartón llega a 0.53, y Vidrio a 0.42, lo que sugiere que estas categorías poseen patrones visuales más distintivos, probablemente gracias a texturas, colores y morfologías más consistentes entre imágenes. En contraste, categorías como Reciclables Varios Madera, Cigarro, Metales y Plásticos exhiben F1-scores muy bajos (entre 0.05 y 0.25), indicando que el modelo tiene dificultades para distinguir las variaciones dentro de estas clases o entre clases visualmente similares.

La matriz de confusión refuerza esta interpretación. Se observan confusiones sistemáticas entre categorías que comparten características visuales o materiales. Por ejemplo, muchas instancias de Reciclables Varios Plástico son clasificadas erróneamente como Vidrio o Textiles, lo que podría deberse a brillos y texturas similares o a un solapamiento en las condiciones de captura de las imágenes. También se detecta una fuerte confusión entre Textiles y “Todo lo demás”, lo cual refleja la amplia heterogeneidad de ambas categorías y la falta de patrones visuales robustos dentro de ellas. Por otro lado, se observa un desempeño muy destacado en la clase de Textiles, con un elevado número de aciertos; esto sugiere que el modelo logró identificar algún rasgo visual dominante en esta clase, posiblemente relacionado con texturas o patrones repetitivos.

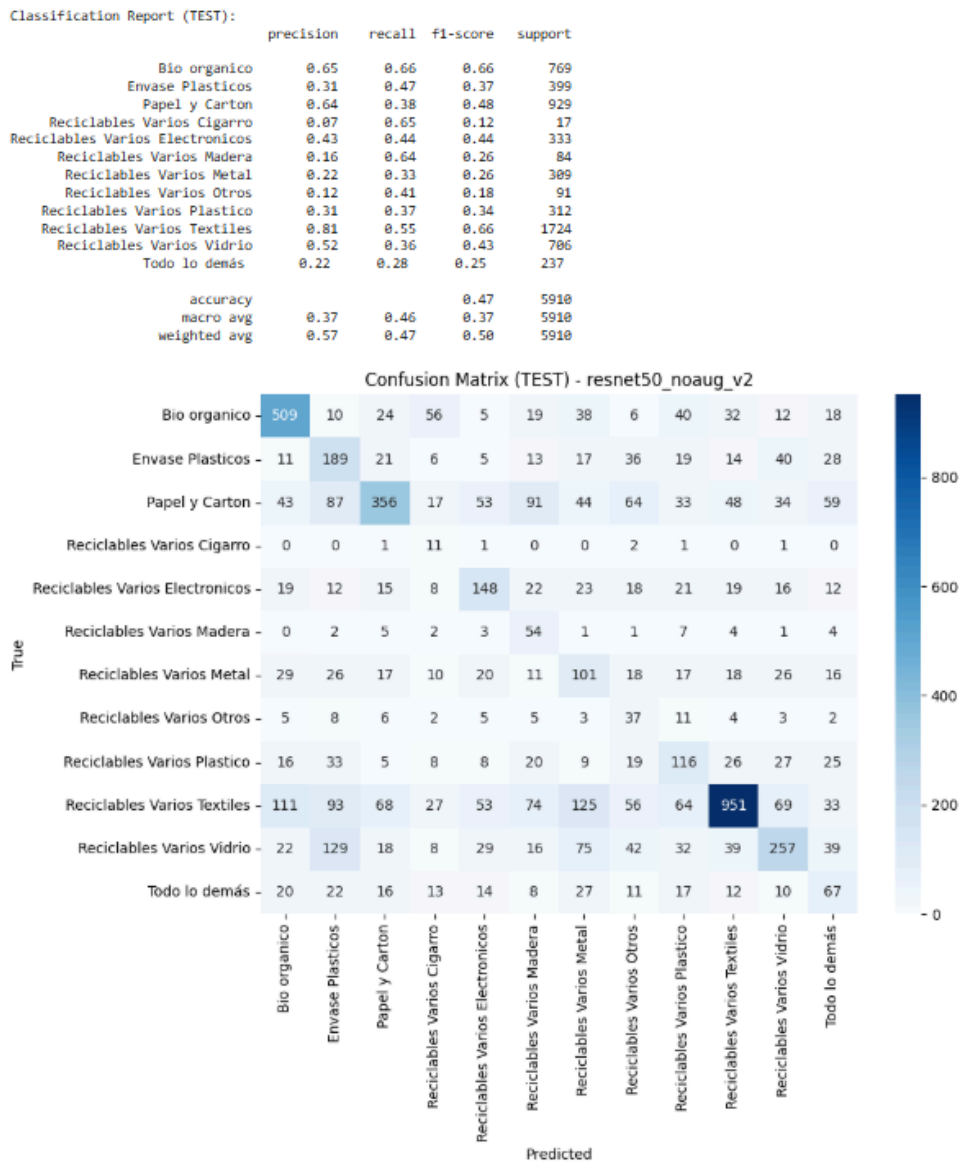


Las curvas de entrenamiento y validación muestran un comportamiento coherente con un modelo que se beneficia del augmentation. La training accuracy aumenta progresivamente a lo largo de las 20 épocas, mientras que la validation accuracy se mantiene consistentemente por encima de la training accuracy. Esto es un patrón común cuando se utiliza augmentation, ya que el modelo ve imágenes más variadas en entrenamiento, lo que limita el sobreajuste. Las curvas de pérdida refuerzan esta conclusión: ambas disminuyen de manera estable, sin un aumento súbito de la validation loss, lo que indica que no se observa sobreajuste severo.

En conjunto, estos resultados muestran que el uso de augmentation ayudó a mejorar la capacidad de generalización del modelo y estabilizar el entrenamiento, pero también pone en evidencia que la clasificación de residuos sigue siendo un problema particularmente complejo

debido a la alta variabilidad intra-clase, las similitudes visuales entre materiales distintos y la falta de rasgos distintivos en algunas categorías. Esto sugiere que futuras mejoras podrían provenir de arquitecturas más robustas (como EfficientNet), técnicas más avanzadas de balanceo de clases, augmentations más específicos por tipo de material o la redefinición de algunas categorías demasiado amplias o ambiguas.

Resultados Resnet50 sin augmentation

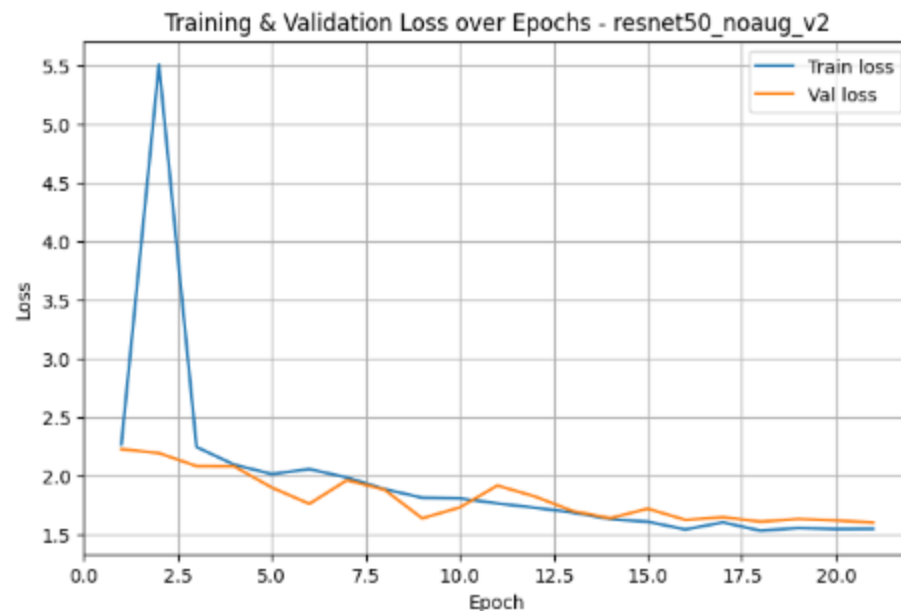
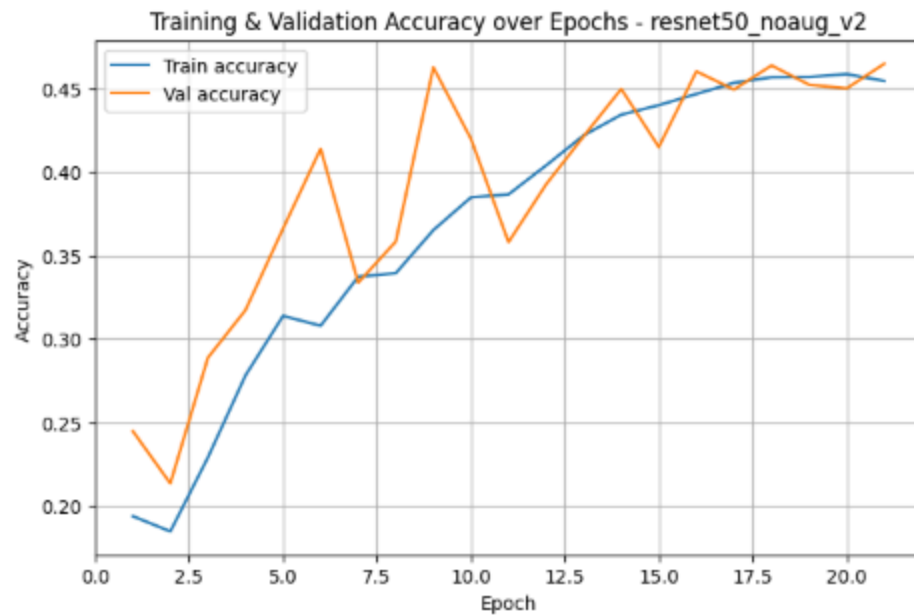


La variante de ResNet50 entrenada sin técnicas de data augmentation constituye un punto de comparación fundamental para evaluar el aporte real de las transformaciones aplicadas durante el entrenamiento. En términos generales, el desempeño alcanzado por este modelo fue

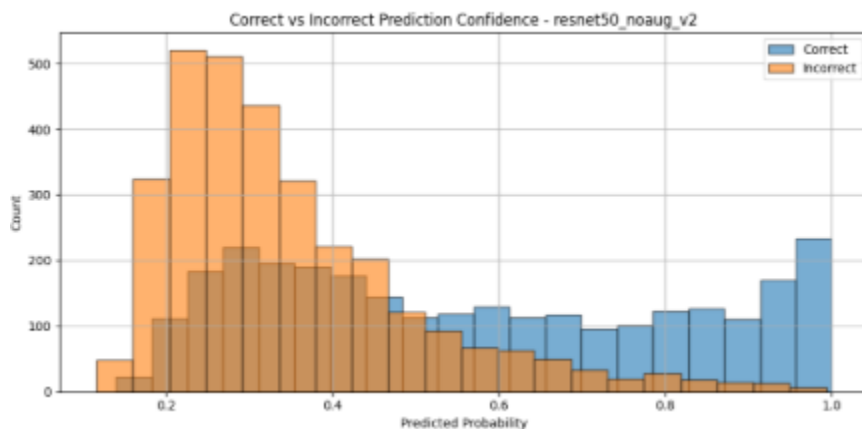
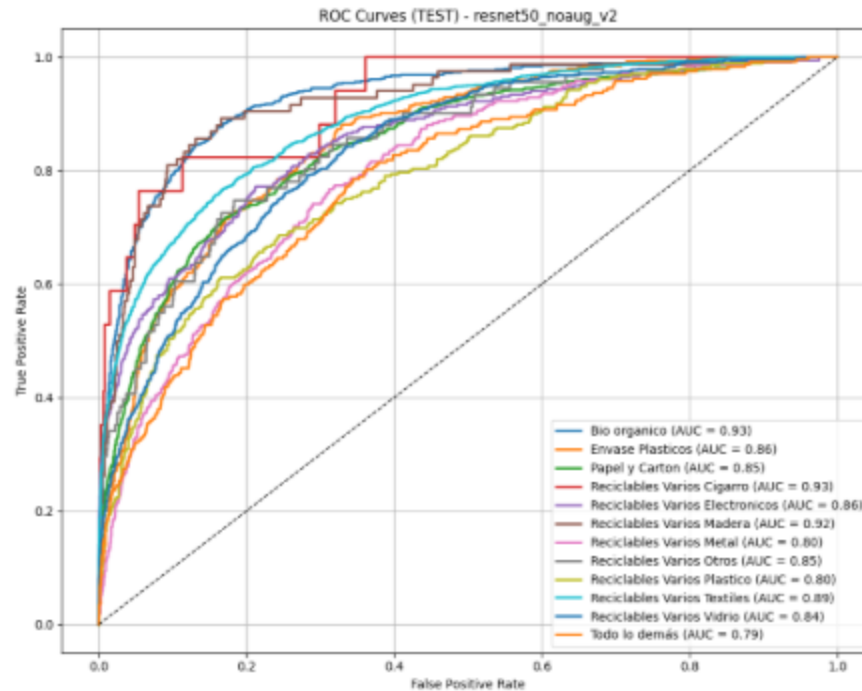
superior al de la versión entrenada con augmentation, lo cual resulta particularmente interesante dado que, en teoría, el aumento artificial de la variabilidad debería favorecer la capacidad de generalización. El modelo sin augmentation obtuvo una accuracy de test de 0.4731, más alta que la obtenida en la variante con augmentation.

El análisis del classification report muestra que varias categorías alcanzaron valores de F1-score significativamente elevados. Clases como Papel y Cartón, Reciclables Varios Textiles y Envase Plásticos presentaron métricas de recall especialmente altas, indicando que el modelo fue capaz de identificar correctamente una proporción considerable de ejemplos verdaderos de estas clases. De forma similar, la categoría Bio orgánico obtuvo un desempeño sólido, con un F1-score de 0.66, lo que evidencia una mejor discriminación respecto a la versión con augmentation.

La matriz de confusión permite observar que, aunque se mantienen algunos patrones de confusión esperables, especialmente entre clases visualmente similares como Reciclables Varios Plásticos y Envase Plásticos, o entre Papel y Cartón y Todo lo demás, el número de aciertos es significativamente mayor en casi todas las categorías respecto al modelo con augmentation. Este comportamiento sugiere que el modelo logró aprovechar mejor la información disponible en las imágenes originales, sin verse afectado por la variabilidad introducida artificialmente.



Las curvas de entrenamiento muestran un patrón de aprendizaje relativamente estable. La pérdida de entrenamiento disminuye de forma sostenida hasta converger alrededor de 1.5, mientras que la pérdida de validación sigue una trayectoria paralela, indicando que el modelo no sufrió sobreajuste severo. Asimismo, tanto la accuracy de entrenamiento como la de validación crecen de forma progresiva y mantienen una brecha controlada, lo cual refuerza la idea de que el modelo fue capaz de aprender representaciones robustas sin necesidad de técnicas de augmentation.



El análisis de las curvas ROC para cada clase muestra resultados consistentes con lo observado previamente. Varias clases alcanzan valores de AUC por encima de 0.85, y categorías como Bio orgánico, Papel y Cartón o Reciclables Varios Textiles se acercan a un comportamiento casi óptimo. Esto indica que el modelo no solo acierta más ejemplos, sino que también asigna probabilidades bien calibradas, distinguiendo adecuadamente entre clases positivas y negativas. Finalmente, la comparación entre las distribuciones de confianza en predicciones correctas versus incorrectas revela que el modelo tiende a asignar mayores probabilidades a predicciones efectivamente correctas, lo cual muestra una calibración razonable incluso sin técnicas adicionales de ajuste.

Comparación entre ResNet50 con y sin augmentation

La comparación entre ambas configuraciones permite resaltar un resultado contraintuitivo: el ResNet50 entrenado sin data augmentation obtuvo mejores métricas globales y por clase que la versión con augmentation. Existen varias posibles razones que explican esta diferencia. En primer lugar, la naturaleza del dataset, con imágenes provenientes de un dominio relativamente controlado y clases bien estructuradas, podría hacer que la introducción de variaciones fuertes durante el entrenamiento genere ruido adicional, dificultando que el modelo aprenda patrones discriminatorios esenciales. En segundo lugar, ciertos tipos de augmentations aplicados (rotaciones, cambios de color o contraste) podrían estar alterando características visuales clave para distinguir clases similares, lo que reduciría la precisión del modelo.

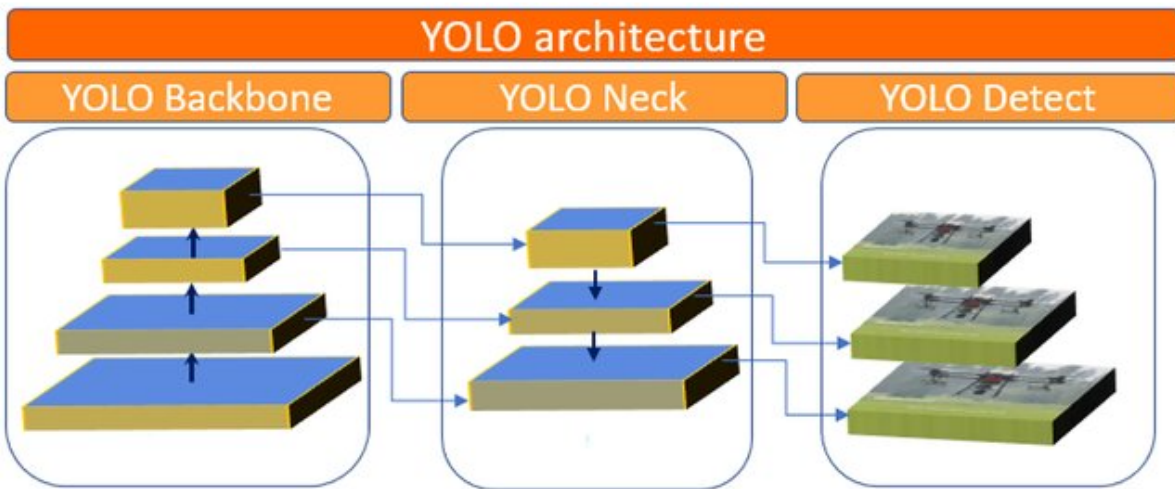
Por el contrario, el modelo sin augmentation pudo concentrarse en aprender directamente de los patrones reales del dataset, logrando una mejor especialización para este dominio particular. Esto se vio reflejado en una mayor accuracy de test, mejores F1-scores para la mayoría de las clases, curvas de aprendizaje más estables y una calibración superior de probabilidades. En síntesis, los resultados sugieren que, para este caso específico, el uso extensivo de data augmentation no resultó beneficioso, y que la versión sin transformations demostró una capacidad de generalización más adecuada al dominio del problema.

EfficientNet - mejora sobre resnet50

Además de ResNet50, se seleccionó la arquitectura EfficientNet como segundo modelo a evaluar. EfficientNet introduce un enfoque sistemático para escalar redes neuronales denominado compound scaling, que ajusta simultáneamente la profundidad, el ancho y la resolución de entrada siguiendo un conjunto equilibrado de coeficientes. Esta estrategia permite obtener modelos más precisos y eficientes en términos de parámetros y operaciones por segundo, en comparación con otras arquitecturas de tamaño equivalente.

EfficientNet ha mostrado resultados buenos en tareas de clasificación de imágenes y ha sido ampliamente adoptado en escenarios donde se requiere un equilibrio entre rendimiento y eficiencia computacional, condiciones que son relevantes para aplicaciones reales de clasificación de residuos, especialmente en dispositivos móviles o sistemas embebidos de bajo consumo. En este proyecto, se entrenan versiones con y sin augmentation siguiendo una metodología equivalente a la aplicada a ResNet50, lo que permitirá comparar ambas arquitecturas en igualdad de condiciones y determinar cuál ofrece el mejor compromiso entre precisión, eficiencia y capacidad de generalización.

YOLO



YOLO (You Only Look Once) es una familia de modelos de visión por computadora diseñada inicialmente para la detección de objetos en tiempo real. YOLO representa un cambio fundamental respecto a las arquitecturas tradicionales de detección, que usualmente procesan la imagen en múltiples etapas (por ejemplo, generando regiones propuestas y posteriormente clasificándolas). En contraste, YOLO plantea un enfoque unificado: realizar todas las predicciones en una sola pasada por la red.

La motivación principal detrás de YOLO fue lograr un detector que fuera simultáneamente rápido, preciso y simple de entrenar. Su arquitectura, basada en una única red convolucional que divide la imagen en cuadrículas y predice bounding boxes y clases directamente, permitió eliminar procesos intermedios costosos y mejorar significativamente la velocidad de inferencia. Gracias a esta simplificación estructural, YOLO se consolidó rápidamente como una de las familias de modelos más utilizadas para aplicaciones en tiempo real, como robótica, vehículos autónomos y sistemas industriales.

A lo largo de sus distintas versiones (YOLOv1 a YOLOv8), la arquitectura ha ido incorporando mejoras en eficiencia, profundidad, regularización y técnicas de entrenamiento. YOLOv8, desarrollado por Ultralytics, integra avances modernos en diseño de redes, como capas C2f, convoluciones optimizadas y backbones tipo CSP evolucionados, logrando una combinación sobresaliente de velocidad, capacidad de generalización y facilidad de uso. Además de la detección, YOLOv8 expandió su alcance ofreciendo variantes para segmentación, pose estimation y clasificación de imágenes, aprovechando la flexibilidad de su backbone.

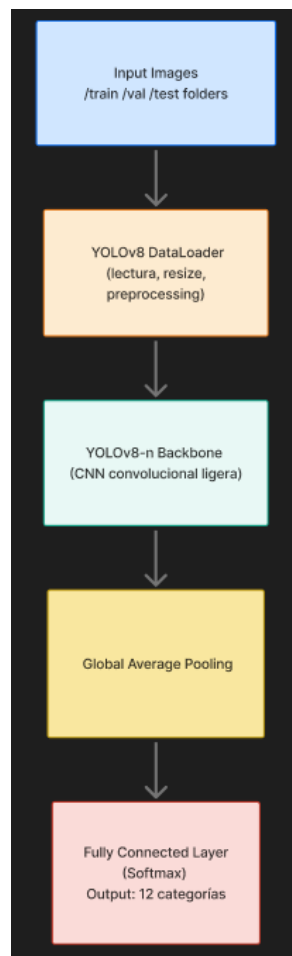
La razón por la cual YOLO es especialmente rápido se debe a tres factores principales:

1. **Inferencia de una sola pasada:** toda la predicción se realiza con un solo forward pass del modelo.

2. **Arquitectura altamente optimizada:** bloques convolucionales ligeros diseñados para maximizar eficiencia.
3. **Paralelización eficiente:** la estructura regular del modelo favorece el aprovechamiento pleno de GPU.

Estas características convierten a YOLOv8 en una alternativa atractiva no solo para tareas de detección, sino también para problemas de clasificación como el abordado en este proyecto, permitiendo explorar un enfoque moderno, liviano y de alto rendimiento dentro del dominio de clasificación de residuos.

Adaptación de YOLOv8 a la Clasificación de Residuos



Además de los modelos basados en arquitecturas tradicionales de clasificación (ResNet50 y EfficientNet), se incorporó como experimento adicional un modelo de la familia YOLOv8, desarrollado por Ultralytics. Si bien YOLO se conoce principalmente como un modelo orientado a la detección de objetos, la versión YOLOv8 incluye también variantes específicas para clasificación de imágenes, permitiendo evaluar su desempeño en un entorno distinto a su uso habitual. Esto hace que sea una opción interesante e innovadora dentro del proyecto, ya que

permite comparar un modelo de arquitectura liviana y altamente optimizada contra modelos clásicos de clasificación.

Modelo base y librería utilizada

El entrenamiento se realizó utilizando la librería oficial Ultralytics (Python), la cual proporciona una API unificada para entrenamiento, evaluación y gestión de modelos YOLO. Esta librería ofrece soporte directo para tareas de clasificación a través de la familia de modelos YOLOv8-cls, los cuales están preentrenados sobre ImageNet e incluyen variantes de diferente tamaño y capacidad computacional.

En nuestro caso, se utilizó el modelo yolov8n-cls.pt (“nano”), que es el más liviano de la familia. Su elección se fundamenta en tres razones principales:

1. permite tiempos de entrenamiento reducidos,
2. es adecuado para trabajar en entornos como Google Colab con recursos limitados, y
3. facilita una primera comparación rápida contra las redes más grandes del proyecto.

Arquitectura YOLOv8 para clasificación

La versión YOLOv8-cls utiliza un backbone convolucional optimizado (basado en la arquitectura CSP y convoluciones modernas tipo C2f), seguido de un head de clasificación que reemplaza la estructura típica de detección. El pipeline del modelo incluye:

- bloques convolucionales livianos optimizados para velocidad,
- etapas de reducción de resolución progresiva,
- un **Global Average Pooling** para condensar la representación espacial,
- una capa fully-connected final con activación softmax.

Esta arquitectura está diseñada para maximizar el rendimiento en tareas de clasificación manteniendo una eficiencia computacional muy alta.

Ajuste del modelo a nuestro caso específico

Para adaptar YOLOv8-cls a la clasificación de residuos, se construyó un dataset siguiendo la estructura de carpetas requerida por Ultralytics para la tarea de clasificación:

```
/train/<clase>/imagen.jpg
```

```
/val/<clase>/imagen.jpg  
/test/<clase>/imagen.jpg
```

Para evitar duplicar las más de 10.000 imágenes disponibles, se generó la estructura utilizando symlinks, lo que permitió mantener un dataset liviano sin comprometer la organización esperada por la librería.

Además, se realizó un split estratificado del dataset manteniendo la distribución de clases en proporciones 80% (train), 10% (validación) y 10% (test). De esta manera, se garantiza una evaluación consistente para todas las categorías del dominio.

Configuración del entrenamiento

El modelo se entrenó con los siguientes hiperparámetros:

- **Épocas:** 20
- **Tamaño de imagen (imgsz):** 224×224, para mantener comparabilidad directa con ResNet50 y EfficientNet
- **Batch size:** 64
- **Workers:** 2
- **Optimizador:** Adam (por defecto)
- **Función de pérdida:** Cross-Entropy (por defecto para clasificación)
- **Evaluación en cada época:** habilitada (val=True)

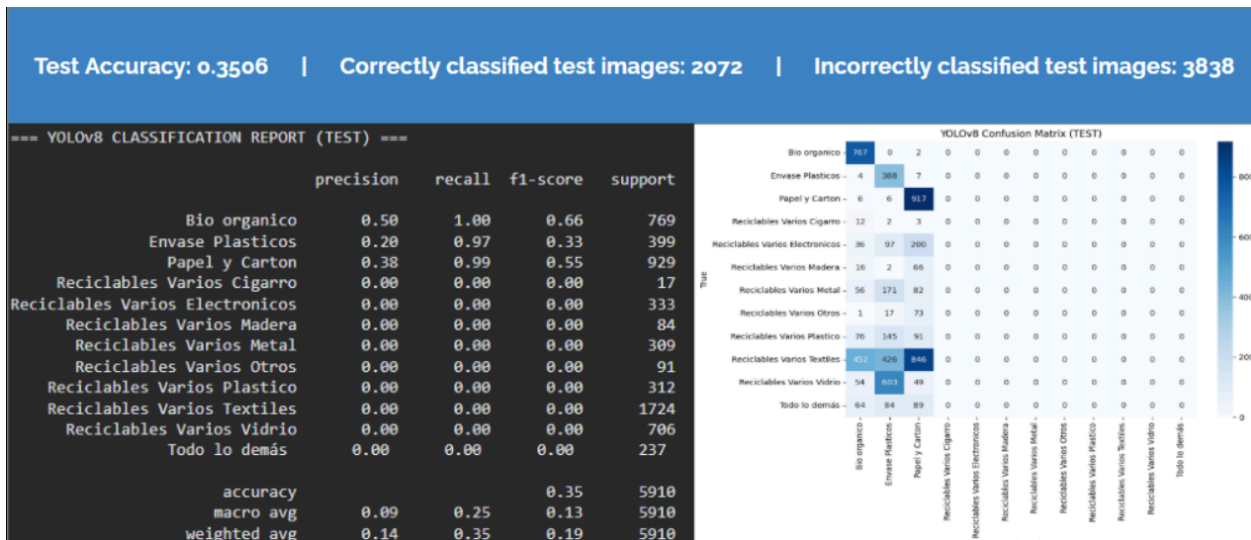
A diferencia del pipeline aplicado a ResNet50, en este caso no se utilizó augmentation manual. La motivación fue obtener un punto de comparación directo con la versión “sin augmentations” de ResNet50, evaluando la robustez intrínseca del modelo YOLO frente a variaciones reales del dataset.

Entrenamiento y ajuste final

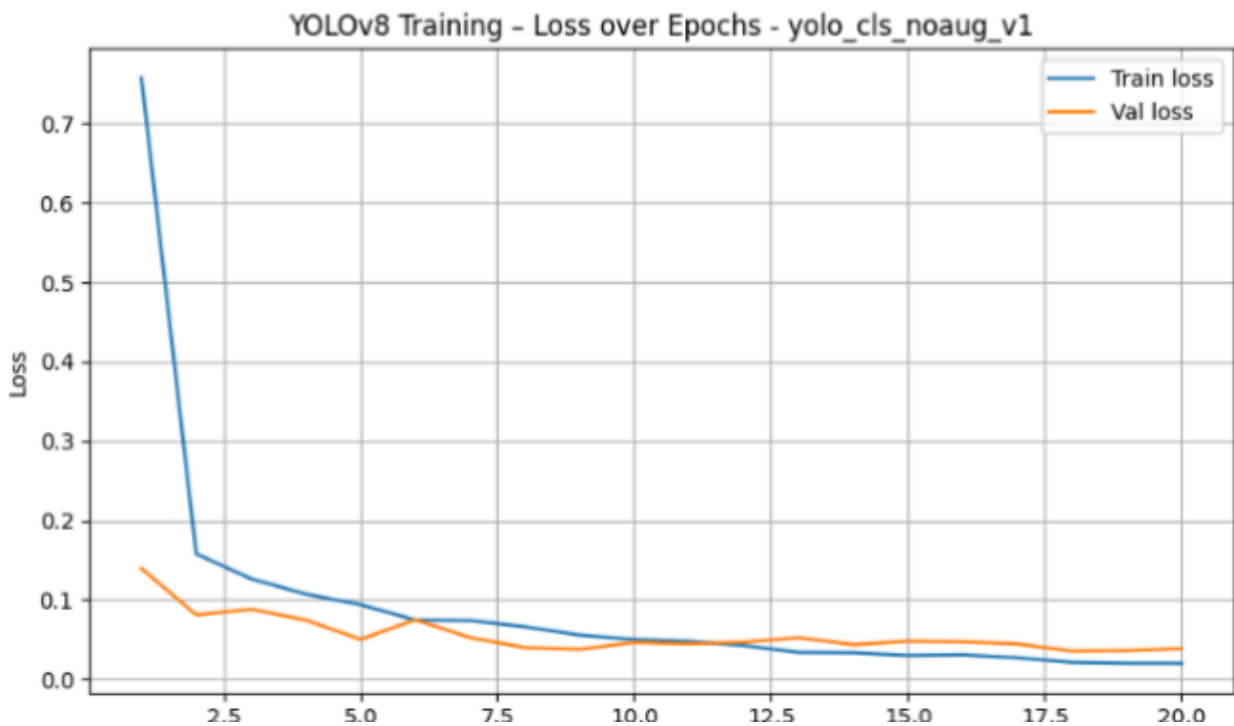
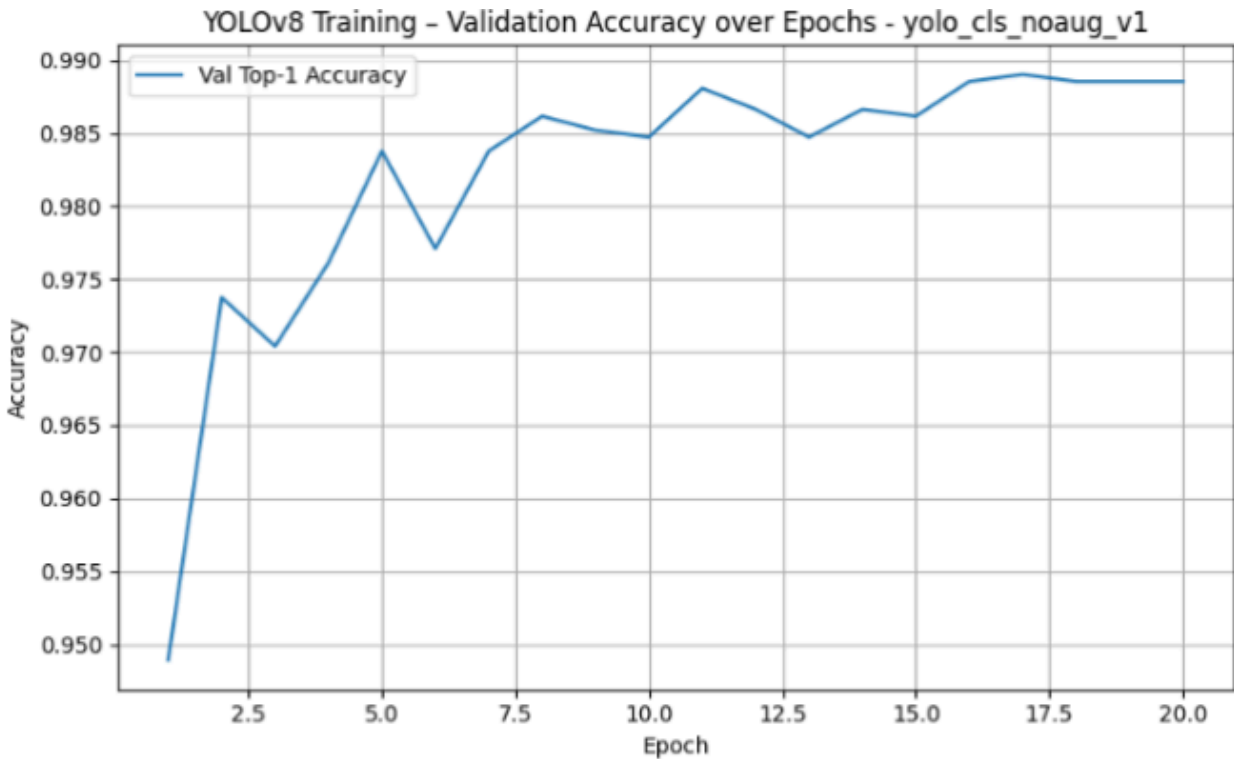
El entrenamiento se realizó íntegramente con el script oficial de Ultralytics, que gestiona automáticamente el preprocesamiento, el pipeline de entrenamiento y la generación de métricas. La única adaptación necesaria consistió en estructurar correctamente el dataset y definir los hiperparámetros.

Tras 20 épocas, el modelo convergió sin problemas y produjo los artefactos necesarios para el análisis posterior: métricas de validación, gráfica de pérdida y del accuracy, y finalmente la evaluación sobre el conjunto de prueba.

Resultados YOLO sin Augmentation

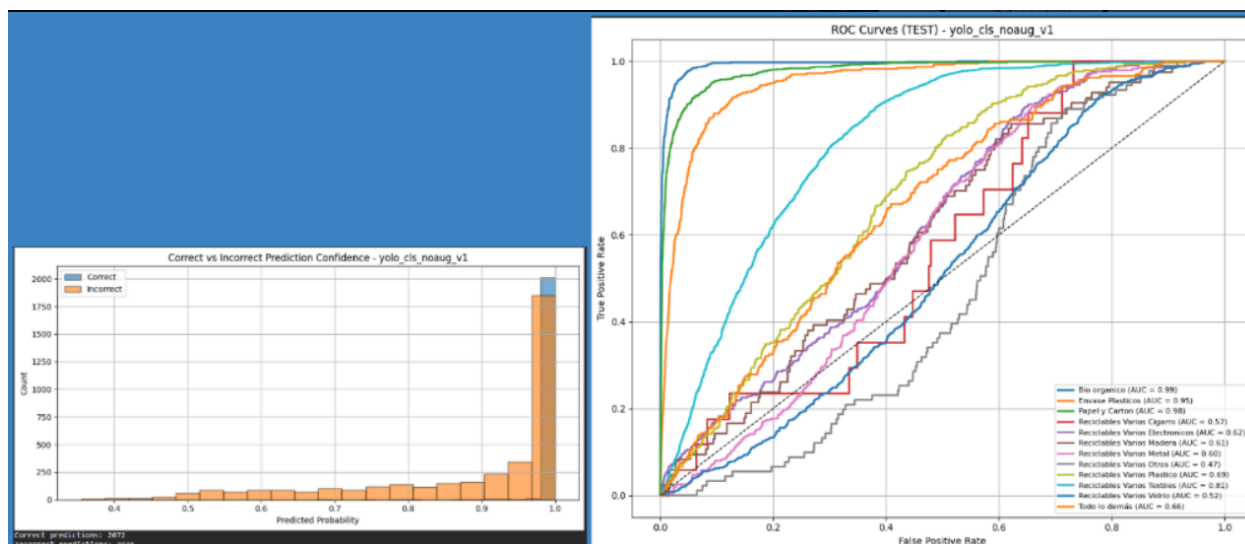


Los resultados del modelo YOLOv8-cls entrenado sin técnicas de augmentación muestran un comportamiento claramente dicotómico entre su desempeño durante el entrenamiento/validación y su capacidad de generalización sobre el conjunto de test. Durante el entrenamiento, las curvas de accuracy y loss indican una convergencia rápida y estable: la accuracy de validación alcanza valores cercanos al 99%, mientras que las pérdidas de entrenamiento y validación descienden de forma sostenida hasta niveles muy bajos. Estas curvas, a primera vista, sugieren un modelo capaz de representar adecuadamente los datos del conjunto de entrenamiento y de validar sus predicciones en muestras no vistas durante el proceso de entrenamiento. Sin embargo, este excelente desempeño contrasta de manera significativa con los resultados obtenidos sobre el conjunto de test.



En el set de test, el modelo obtiene una accuracy global de apenas 0.3506, clasificando correctamente 2.072 imágenes de un total de 5.910. Esta diferencia tan marcada entre validación y test es un indicador fuerte de overfitting, probablemente potenciado por la ausencia total de augmentación. El modelo parece haber aprendido patrones muy específicos de las imágenes utilizadas durante el entrenamiento y la validación, pero no logra generalizar

adecuadamente cuando se enfrenta a imágenes nuevas. La alta performance en validación probablemente se explica por una distribución visual muy similar entre train y val, mientras que el test presenta variaciones intra-clase más significativas (distintos ángulos, condiciones de iluminación, tipos de residuos, fondos, contaminaciones, etc.).



El análisis del classification report y de la matriz de confusión confirma esta falta de generalización. El modelo muestra un desempeño aceptable solo en unas pocas clases, como “Bio orgánico”, donde alcanza un recall de 1.00 y un f1-score de 0.66. Esta clase suele presentar patrones visuales distintivos (restos orgánicos marrones/verdosos, formas irregulares), lo cual facilita el aprendizaje incluso sin augmentation. Otras clases con algo de desempeño, como “Envase plástico” o “Papel y cartón”, muestran recall por encima de 0.85, aunque con precision baja. Esto indica que el modelo frecuentemente elige estas clases como predicción por defecto, generando muchos falsos positivos. Sin embargo, la gran mayoría de las demás clases colapsan por completo, con precision, recall y f1-score iguales a 0. Esto implica que el modelo no logra capturar patrones discriminativos para categorías que visualmente son más complejas, con formas variadas o altos niveles de contaminación visual (por ejemplo, “Reciclables Varios Electrónicos”, “Madera”, “Vidrio”, “Textiles”).

Las curvas ROC aportan un matiz adicional: algunas clases presentan valores de AUC relativamente altos a pesar de que las métricas basadas en clasificación discreta son muy bajas. Esto indica que el modelo sí aprende ciertos patrones útiles pero no logra separar las clases de manera lo suficiente clara como para tomar decisiones correctas con un único umbral. En otras palabras, el modelo tiene cierta capacidad de distinguir entre clases a nivel probabilístico, pero la falta de robustez lo lleva a confundir frecuentemente las categorías con mayor solapamiento visual.

Finalmente, al analizar el histograma de probabilidades de predicción para aciertos y errores, se observa una tendencia preocupante: muchas predicciones incorrectas son realizadas con

alta confianza (valores cercanos a 1.0). Esto es un síntoma típico de overfitting severo. El modelo no solo se equivoca mucho, sino que además lo hace con convicción. Esto indica que las características aprendidas durante el entrenamiento no se corresponden con representaciones generalizables del problema de clasificación de residuos.

En conjunto, estos resultados muestran que YOLOv8-cls, cuando es entrenado sin ninguna técnica de augmentación, no logra generalizar adecuadamente en un problema con alta variabilidad intra-clase, como lo es la clasificación de residuos reales. El modelo evidencia overfitting estructural, alta confianza mal calibrada y una fuerte dependencia de patrones superficiales presentes en el conjunto de entrenamiento. Estos hallazgos no solo justifican la necesidad de aplicar augmentation, sino que también resaltan la complejidad intrínseca del dominio y la importancia de diseñar estrategias de robustecimiento que permitan capturar la variabilidad natural de las imágenes de residuos.

Resultados comparativos - Tabla si se puede: por categoria, modelo, val_accuracy

Conclusiones

El trabajo permitió explorar distintas arquitecturas de visión por computadora aplicadas al problema de clasificación de residuos, un dominio caracterizado por una elevada variabilidad intra-clase, fuertes diferencias en condiciones de iluminación, variedad de fondos y un alto nivel de ruido visual. A partir de esta experimentación, se observa que los modelos basados en CNN tradicionales como ResNet50 ofrecen un comportamiento sólido y predecible, especialmente cuando se los entrena con un pipeline adecuado de augmentación. La comparación entre entrenamiento con y sin augmentación mostró que estas técnicas no solo mejoran la generalización, sino que también reducen considerablemente el overfitting, lo cual es fundamental en datasets donde las clases presentan heterogeneidad visual significativa.

En cuanto a YOLOv8-cls, el modelo mostró resultados contrastantes. Durante el entrenamiento sin augmentación alcanzó métricas de validación excepcionalmente altas, pero su performance en el conjunto de test reveló una generalización muy limitada. Este comportamiento confirma que la arquitectura, aunque eficiente y veloz, requiere obligatoriamente augmentations robustas para evitar el aprendizaje de patrones superficiales. Aun así, los análisis como curvas ROC y distribuciones de confianza sugieren que YOLOv8-cls es capaz de aprender características útiles, lo cual abre la puerta a futuras mejoras mediante ajustes de hiperparámetros, regularización y estrategias de enriquecimiento del dataset.

En conjunto, los experimentos muestran que la clasificación de residuos es un problema desafiante en el cual los modelos no solo deben reconocer objetos, sino también adaptarse a la diversidad morfológica y contextual de los residuos reales. Los resultados indican que, con los recursos disponibles, las arquitecturas clásicas como ResNet50 ofrecen una base sólida,

mientras que modelos más recientes y livianos como YOLOv8-cls requieren un preprocesamiento y una calibración más cuidadosa. Aun así, ambos enfoques demuestran que el uso de visión por computadora es una herramienta viable y prometedora para sistemas automáticos de separación y reciclaje.

Aspectos a mejorar y trabajo futuro

Existen diversas líneas de mejora que pueden potenciar la performance y la robustez de los modelos utilizados. En primer lugar, es necesario ampliar y balancear el dataset. Varias clases presentan pocos ejemplos o alta variabilidad interna, lo que dificulta el aprendizaje de patrones estables. Una mayor recolección de imágenes reales, así como la limpieza y estandarización de las existentes, permitiría entrenar modelos más consistentes. Asimismo, incorporar un pipeline de augmentación más real, incluyendo perturbaciones geométricas, fotométricas y de contexto, es especialmente crítico para modelos como YOLOv8-cls, cuya sensibilidad a la variabilidad visual quedó en evidencia.

Otra línea de mejora involucra la experimentación con arquitecturas más modernas o adaptadas al dominio. Por ejemplo, variantes de EfficientNet o modelos basados en Vision Transformers podrían ofrecer mejor capacidad de representación en escenarios con alta diversidad visual. También sería valioso explorar técnicas de regularización como label smoothing, dropout más elevado, early stopping o la utilización de optimizadores adaptativos distintos a Adam por defecto.

A nivel metodológico, incorporar métricas más allá de la accuracy global, como F1 ponderado o métricas específicas por clase, permitiría evaluar mejor el desempeño en clases minoritarias o difíciles. Del mismo modo, una calibración de probabilidades facilitaría la interpretación de los outputs, especialmente para modelos que mostraron alta confianza en predicciones incorrectas.

Finalmente, un siguiente paso natural sería integrar los modelos en un pipeline de inferencia realista, utilizando imágenes tomadas en diferentes entornos y bajo condiciones no controladas. Esto permitiría evaluar la utilidad práctica del sistema y detectar sesgos o fallas que no siempre emergen en los experimentos controlados. En conclusión, si bien los resultados obtenidos constituyen un avance significativo, existe un amplio margen para extender y robustecer el enfoque, especialmente considerando la aplicación real en sistemas de clasificación automatizada de residuos.

Anexo/Recursos (links a articulos , diagramas, google collab, repositorios importantes)

Clasificaciones:

[Celebramos el Día Mundial del Reciclaje | Portal institucional](#)

ResNet 50

- <https://towardsdatascience.com/the-annotated-resnet-50-a6c536034758/1512.03385>
- [resnet50 — Torchvision main documentation](#)
- [ResNet and ResNetV2](#)
- [keras/keras/src/applications/resnet.py at v3.12.0 · keras-team/keras](#)
- [An Improved ResNet50 for Environment Image Classification - ScienceDirect](#)
- notebook de codigo referencia:
<https://www.kaggle.com/code/rajgupta2019/waste-classification-resnet50>
- What and why to freeze layers in the model:
<https://www.exxactcorp.com/blog/deep-learning/guide-to-freezing-layers-in-ai-models>
- nuestro diagrama de resNet 50 para waste clasiffication:
[https://www.figma.com/board/zO7KGsJNZ6vkw8l6CEBHsL/ResNet-50-Waste-Clasificati](https://www.figma.com/board/zO7KGsJNZ6vkw8l6CEBHsL/ResNet-50-Waste-Clasificati?node-id=0-1&t=DgvwoaQdKCIZsHxV-1)
[on?node-id=0-1&t=DgvwoaQdKCIZsHxV-1](https://www.figma.com/board/zO7KGsJNZ6vkw8l6CEBHsL/ResNet-50-Waste-Clasificati?node-id=0-1&t=DgvwoaQdKCIZsHxV-1)

YOLO

- <https://www.datacamp.com/blog/yolo-object-detection-explained>
- <https://github.com/ultralytics/ultralytics/blob/main/ultralytics/cfg/models/v8/yolov8.yaml>
- <https://docs.ultralytics.com/models/yolov8/>
-